Lab.6

Preprocesor jezyka C

1.

Preprocesor to etap, który działa **przed właściwą kompilacją** programu. Przetwarza on kod źródłowy czysto tekstowo wykonuje polecenia zaczynające się od znaku #, takie jak wstawianie zawartości plików, podstawianie makr czy usuwanie lub dołączanie fragmentów kodu. Preprocesor nie rozumie języka C (nie zna typów ani składni) on tylko zamienia tekst. Dopiero jego wynik trafia do kompilatora.

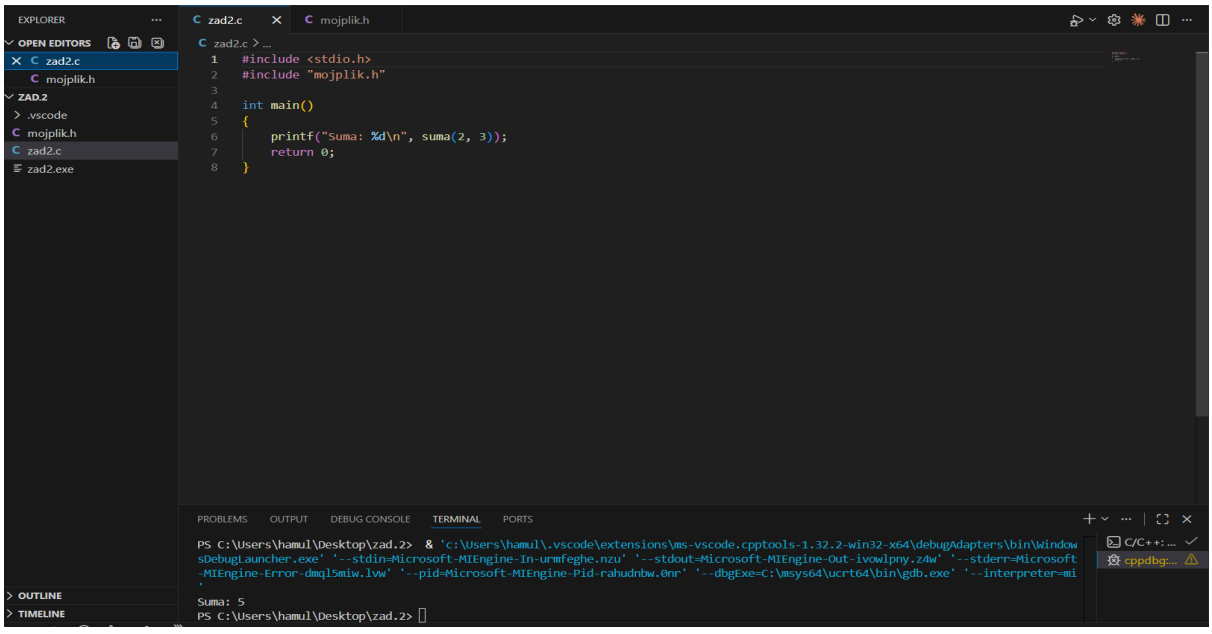
Typowe dyrektywy preprocesora:

Dyrektywa	Zastosowanie
#include	dołączenie zawartości innego pliku (nagłówka)
#define	definicja makra lub stałej symbolicznej
#undef	usunięcie wcześniej zdefiniowanego makra
#if / #elif / #else / #endif	kompilacja warunkowa na podstawie wyrażenia
#ifdef / #ifndef	sprawdzenie, czy makro jest (nie)zdefiniowane
#pragma	instrukcje specyficzne dla kompilatora

2.

#include <nazwa> preprocesor szuka pliku w **katalogach systemowych** (standardowe biblioteki kompilatora). Używamy tej formy dla nagłówków biblioteki standardowej, np. <stdio.h>, <stdlib.h>.

#include "nazwa" preprocesor najpierw szuka pliku w **katalogu projektu** (tam, gdzie znajduje się plik źródłowy), a dopiero potem w katalogach systemowych. Używamy tej formy dla **własnych plików nagłówkowych**, np. "macros.h".**Przykład:** w pliku zad2.c użyto obu form – #include <stdio.h> dla biblioteki standardowej oraz #include "mojplik.h" dla własnego nagłówka z funkcją suma().



3.

Program wypisze **10**. Dyrektywa #define X 10 tworzy stałą symboliczną. Przed kompilacją preprocesor zamienia każde wystąpienie X na 10, więc do kompilatora trafia już printf("%d\n", 10);.

4.

Naiwna definicja #define SQUARE(x) x * x **NIE działa poprawnie dla wyrażeń.** Preprocesor podstawia tekst dosłownie, więc SQUARE(3+1) rozwija się do 3 + 1 * 3 + 1. Z powodu kolejności działań (mnożenie przed dodawaniem) daje to **3 + 3 + 1 = 7**, a nie oczekiwane 16.

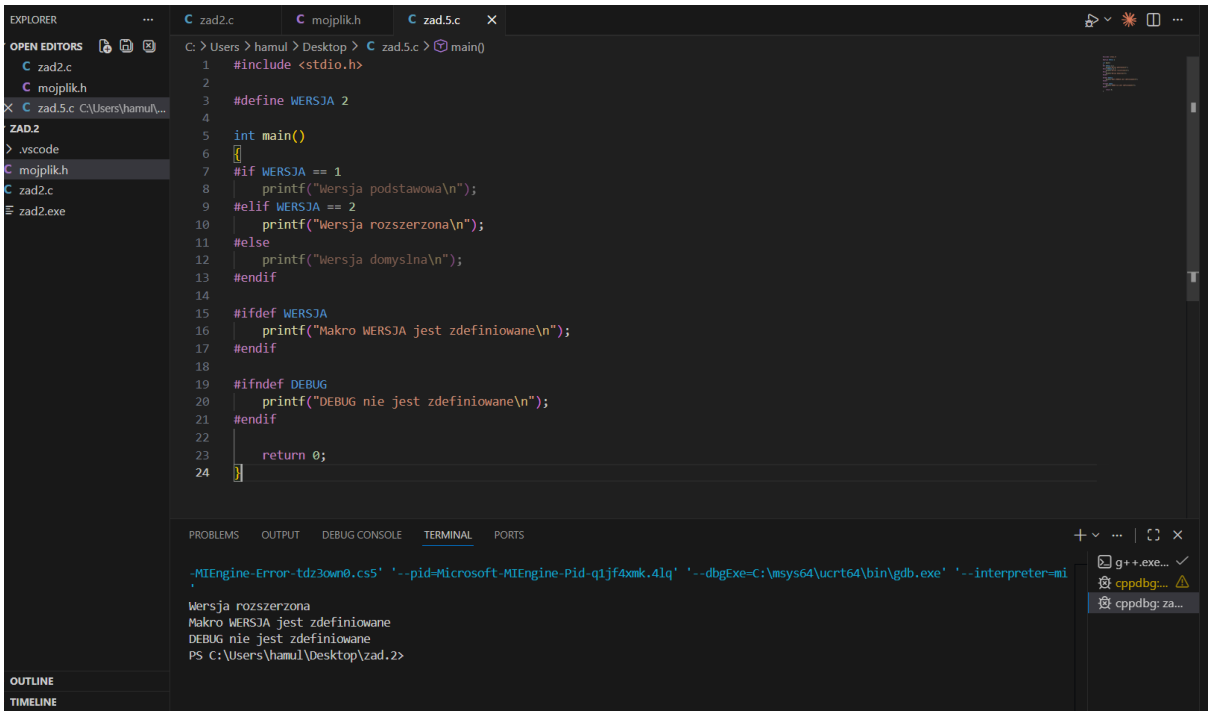
Rozwiązanie: trzeba otoczyć nawiasami zarówno argument, jak i cały wynik: #define SQUARE(x) ((x) * (x)). Wtedy SQUARE(3+1) rozwija się do ((3+1) * (3+1)) = 16. To pokazuje złotą zasadę makr: zawsze otaczaj argumenty nawiasami.

5.

Dyrektywy warunkowe decydują, **które fragmenty kodu w ogóle trafią do kompilatora** (to dzieje się przed kompilacją, a nie w trakcie działania programu).

Dyrektywa	Działanie
#if / #elif / #else / #endif	kompiluje blok zależnie od wartości wyrażenia (np. #if WERSJA == 2)
#ifdef MAKRO	kompiluje blok, jeśli makro JEST zdefiniowane
#ifndef MAKRO	kompiluje blok, jeśli makro NIE jest zdefiniowane

Przykład: w pliku zad5.c ze względu na #define WERSJA 2 kompilowana jest tylko gałąź #elif WERSJA == 2, dlatego program wypisuje „Wersja rozszerzona”, potwierdza zdefiniowanie WERSJA i brak DEBUG.



6.

#undef MAX usuwa wcześniejszą definicję makra MAX. Po tej operacji nazwa staje się „wolna” i można ją zdefiniować na nowo bez ostrzeżenia o redefinicji. W przykładzie MAX najpierw wynosi 100, a po #undef i ponownym #define 200.

Kiedy przydatne: gdy chcemy lokalnie zmienić wartość makra, uniknąć konfliktu nazw między dołączanymi plikami, albo celowo „wyłączyć” makro w pewnym fragmencie kodu.

7.

#define MAX 100 to makro preprocesora zwykłe podstawienie tekstu, wykonywane przed kompilacją. Nie ma typu ani adresu w pamięci. const int MAX = 100 to prawdziwa zmienna z typem, obsługiwana przez kompilator, której nie można zmienić po inicjalizacji.

#define MAX 100	**const int MAX = 100**
Zalety: brak pamięci, działa wszędzie (też w #if), prosty.	Zalety: ma typ (kontrola typów), widoczny w debuggerze, ma zasięg.
Wady: brak typu, brak kontroli zasięgu, trudniejsze debugowanie, ryzyko błędów tekstowych.	Wady: zajmuje pamięć, w czystym C nie zawsze użyje się jej jako rozmiaru tablicy globalnej.

8.

Include guard (strażnik dołączeń) to zabezpieczenie chroniące przed **wielokrotnym dołączeniem** tego samego pliku nagłówkowego. Bez niego, jeśli plik zostanie dołączony dwa razy (np. pośrednio przez inne nagłówki), powstaną błędy „redefinicji”.

Mechanizm: przy pierwszym dołączeniu #ifndef STUDENT_H jest prawdziwe, więc definiujemy STUDENT_H i resztę pliku. Przy kolejnym dołączeniu makro już istnieje, więc cała zawartość między #ifndef a #endif zostaje pominięta.

Przykład: plik student.h zawiera strukturę Student owiniętą w #ifndef STUDENT_H / #define STUDENT_H / ... / #endif.
KOD:

```
#ifndef STUDENT_H
#define STUDENT_H
struct Student
{
   char imie[50];
   int wiek;
   double srednia;
};
#endif
```

9.

Sama logika makr jest poprawna: DEBUG jest zdefiniowane, więc LOG(x) rozwija się do printf("LOG: %s\n", x). Gdyby DEBUG nie było, LOG(x) stałoby się puste (nic by nie robiło).

Błąd: brakuje #include <stdio.h>. Makro LOG wywołuje printf, ale bez dołączenia nagłówka funkcja nie jest zadeklarowana. Kompilator zgłasza ostrzeżenie „implicit declaration of function 'printf'". Brakuje też return 0; w main.

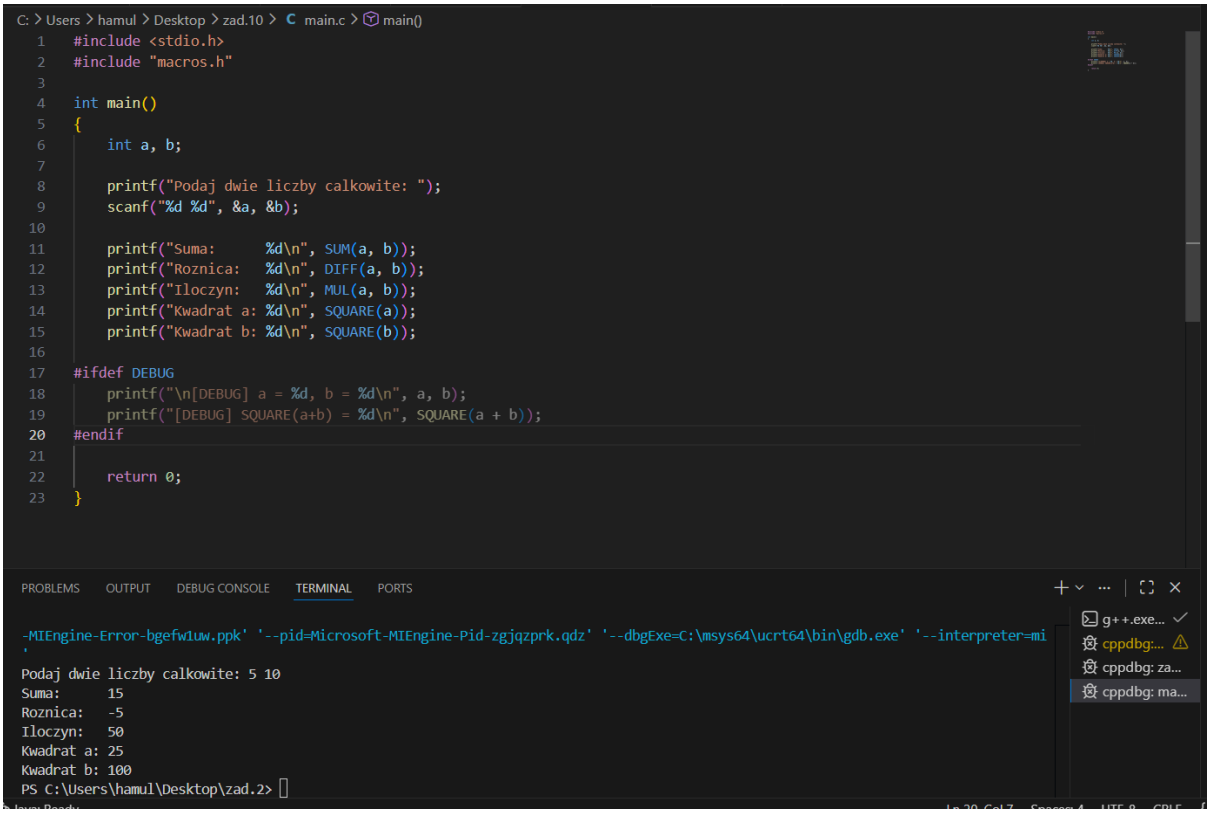
Naprawa: dodać #include <stdio.h> na początku pliku. Wtedy program poprawnie wypisuje „LOG: Test".

10.

Rozwiązanie składa się z dwóch plików. macros.h zawiera cztery makra, każde z pełnym nawiasowaniem, np. #define SUM(a,b) ((a)+(b)) oraz #define SQUARE(x) ((x)*(x)) dzięki temu działają poprawnie nawet dla wyrażeń typu SQUARE(3+1).

Plik main.c dołącza "macros.h", wczytuje dwie liczby przez scanf i wypisuje sumę, różnicę, iloczyn oraz kwadraty. Sekcja w #ifdef DEBUG wypisuje dodatkowe informacje. Włącza się ją przy kompilacji flagą gcc -DDEBUG main.c (bez zmiany kodu źródłowego).

Czy makra mają sens? Dla tak prostych operacji raczej nie. Makra nie sprawdzają typów i mogą powodować błędy (np. podwójne obliczenie argumentu albo problem z brakiem nawiasów). Lepszym wyborem są funkcje inline lub zwykłe funkcje, które są bezpieczniejsze i czytelniejsze. Makra mają sens głównie tam, gdzie potrzebna jest niezależność od typu lub działanie na etapie preprocesora.



11.

Ponieważ wybór następuje na etapie preprocesora, **ten sam kod kompiluje się poprawnie na obu systemach** kompilator widzi tylko gałąź pasującą do swojej platformy. Dodatkowo własny symbol #define TEST uaktywnia blok #ifdef TEST, który wypisuje „Tryb testowy włączony”.

